`workflow` keyword

Tier: Free, Premium, Ultimate

Offering: GitLab.com, GitLab Self-Managed, GitLab Dedicated

Use the `workflow` keyword in your `.gitlab-ci.yml` file to control when pipelines are created.

The `workflow` keyword is evaluated before jobs. For example, if a job is configured to run for tags, but the workflow prevents tag pipelines, the job never runs.

Common `if` clauses for `workflow:rules`

Some example `if` clauses for `workflow: rules`:

Example rules	Details
`if: '$CI_PIPELINE_SOURCE == "merge_request_event"'`	Control when merge request pipelines run.
`if: '$CI_PIPELINE_SOURCE == "push"'`	Control when both branch pipelines and tag pipelines run.
`if: $CI_COMMIT_TAG`	Control when tag pipelines run.
`if: $CI_COMMIT_BRANCH`	Control when branch pipelines run.

See the common `if` clauses for `rules` for more examples.

`workflow: rules` examples

In the following example:

- Pipelines run for all `push` events (changes to branches and new tags).
- Pipelines for push events with commit messages that end with `-draft` don't run, because they are set to `when: never`.
- Pipelines for schedules or merge requests don't run either, because no rules evaluate to true for them.

```yaml
workflow:
  rules:
    - if: $CI_COMMIT_MESSAGE =~ /-draft$/
      when: never
    - if: $CI_PIPELINE_SOURCE == "push"
```

This example has strict rules, and pipelines do **not** run in any other case.

Alternatively, all of the rules can be `when: never`, with a final `when: always` rule. Pipelines that match the `when: never` rules do not run. All other pipeline types run. For example:

```yaml
```

```
    when: never
  - if: $CI_PIPELINE_SOURCE == "push"
    when: never
  - when: always
```

This example prevents pipelines for schedules or `push` (branches and tags) pipelines. The final `when: always` rule runs all other pipeline types, **including** merge request pipelines.

Switch between branch pipelines and merge request pipelines

To make the pipeline switch from branch pipelines to merge request pipelines after a merge request is created, add a `workflow: rules` section to your `.gitlab-ci.yml` file.

If you use both pipeline types at the same time, duplicate pipelines might run at the same time. To prevent duplicate pipelines, use the `CI_OPEN_MERGE_REQUESTS` variable.

The following example is for a project that runs branch and merge request pipelines only, but does not run pipelines for any other case. It runs:

- Branch pipelines when a merge request is not open for the branch.
- Merge request pipelines when a merge request is open for the branch.

yaml

```yaml
workflow:
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH && $CI_OPEN_MERGE_REQUESTS
      when: never
    - if: $CI_COMMIT_BRANCH
```

If GitLab attempts to trigger:

- A merge request pipeline, start the pipeline. For example, a merge request pipeline can be triggered by a push to a branch with an associated open merge request.
- A branch pipeline, but a merge request is open for that branch, do not run the branch pipeline. For example, a branch pipeline can be triggered by a change to a branch, an API call, a scheduled pipeline, and so on.
- A branch pipeline, but there is no merge request open for the branch, run the branch pipeline.

You can also add a rule to an existing `workflow` section to switch from branch pipelines to merge request pipelines when a merge request is created.

Add this rule to the top of the `workflow` section, followed by the other rules that were already present:

yaml

```yaml
workflow:
  rules:
    - if: $CI_COMMIT_BRANCH && $CI_OPEN_MERGE_REQUESTS && $CI_PIPELINE_SOURCE == "push"
      when: never
    - # Previously defined workflow rules here
```

Triggered pipelines that run on a branch have a `$CI_COMMIT_BRANCH` set and could be blocked by a similar rule. Triggered pipelines have a pipeline source of `trigger` or `pipeline`, so `&& $CI_PIPELINE_SOURCE == "push"` ensures the rule does not block triggered pipelines.

feature branches, while keeping long-lived branches to support multiple versions of your software.

For example, to only run pipelines for your merge requests, tags, and protected branches:

```yaml
workflow:
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_TAG
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
    - if: $CI_COMMIT_REF_PROTECTED == "true"
```

This example assumes that your default branch or other long-lived branches are protected.

Skip pipelines for draft merge requests

You can use `workflow: rules` to skip pipelines for draft merge requests. This approach saves compute resources until development is complete.

Use the `CI_MERGE_REQUEST_DRAFT` variable to check if a merge request is in draft state. This variable automatically detects all draft formats that GitLab supports.

```yaml
workflow:
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event" && $CI_MERGE_REQUEST_DRAFT == "true"
      when: never
    - when: always

stages:
  - build

build-job:
  stage: build
  script:
    - echo "Testing"
```

> The `CI_MERGE_REQUEST_DRAFT` variable was introduced in GitLab 17.10. For earlier versions, use `CI_MERGE_REQUEST_TITLE` with a regular expression instead.

Troubleshooting

Merge request stuck with `Checking pipeline status.` message

If a merge request displays `Checking pipeline status.`, but the message never goes away (the “spinner” never stops spinning), it might be due to `workflow:rules`. This issue can happen if a project has **Pipelines must succeed** enabled, but the `workflow:rules` prevent a pipeline from running for the merge request.

For example, with this workflow, merge requests cannot be merged, because no pipeline can run:

```yaml
```

```
      - .gitlab/**/**.md
    when: never
```

BY-SA